

Những tư tưởng và phương pháp cơ bản của lý thuyết đồ thị

Ren Kai

Trường Trung học Changjun, tỉnh Hồ Nam

2005

Nguồn gốc: Những tư tưởng và phương pháp cơ bản của lý thuyết đồ thị

IOI China candidate team papers / 2005 / Ren Kai.doc

Tóm tắt

Bài viết tập trung thảo luận các tư tưởng và phương pháp cơ bản của lý thuyết đồ thị, không đi vào các thuật toán đồ thị cao sâu.

Bài viết chủ yếu trình bày tư tưởng cơ bản của lý thuyết đồ thị từ hai phía: một là chọn hợp lý mô hình đồ thị; hai là đào sâu bản chất bài toán và tận dụng đầy đủ các đặc tính của mô hình. Đồng thời bài viết cũng tổng kết một số phương pháp có tính phổ quát khi giải bài toán.

Từ khóa: tư tưởng cơ bản; mô hình đồ thị; bản chất bài toán; phương pháp định nghĩa; phương pháp phân tích; phương pháp tổng hợp.

1 Dẫn nhập

Đồ thị dùng các đỉnh và các cạnh để mô tả sự vật và quan hệ giữa các sự vật; nó là một dạng trừu tượng hóa của bài toán thực tế. Sở dĩ ta dùng đồ thị để giải bài toán là vì đồ thị có thể biến thông tin rối rắm thành có trật tự, trực quan và rõ ràng. Vì vậy tư tưởng cơ bản nhất trong lý thuyết đồ thị là xây dựng một mô hình thích hợp, đào sâu bản chất bài toán, phân tích và sử dụng các tính chất của mô hình đồ thị, từ đó đạt mục đích giải bài toán. Dưới đây ta sẽ tập trung vào việc chọn mô hình và khai thác tính chất của đồ thị để trình bày tư tưởng cơ bản và phương pháp vận dụng đồ thị.

2 Chọn hợp lý mô hình đồ thị

Khi giải một bài toán thực tế, ta thường trước hết trừu tượng hóa bài toán ấy thành một mô hình toán học, sau đó tìm phương pháp thích hợp trên mô hình, cuối cùng đưa phương pháp ấy trở lại bản thân bài toán thực tế. Mô hình đồ thị là một loại mô hình toán học đặc biệt. Khi xây dựng mô hình đồ thị, ta dùng đỉnh và cạnh của đồ thị để thể hiện đặc điểm của bài toán gốc. Mô hình được xây dựng nhất thiết phải phản ánh bản chất bài toán một cách chân thực, phù hợp và thấu đáo, đồng thời cũng phải cố gắng ngắn gọn, rõ ràng. Các bài toán đồ thị thường có quan hệ chằng chịt và biến hóa đa dạng, nên xây dựng được một mô hình thích hợp thật không dễ. Khi chọn mô hình đồ thị, cần phân tích sâu đặc điểm của bài toán thực tế, mạnh dạn phỏng đoán và kiểm chứng. Dưới đây, thông qua một ví dụ cụ thể, ta thấy được tầm quan trọng của việc chọn đúng mô hình đồ thị và một số phương pháp liên quan.

2.1 Ví dụ 1: Người trượt tuyết

Nguồn bài. Poland Olympiad of Informatics 2002, vòng III: *Skiers*.

Tóm lược đề bài. Cho một đồ thị phẳng gồm n đỉnh, $2 \leq n \leq 5000$, và m cạnh có hướng. Mỗi đỉnh có hoành độ và tung độ khác nhau; có một đỉnh cao nhất v_h và một đỉnh thấp nhất v_l . Mỗi cạnh có hướng nối hai đỉnh khác nhau, chiều cạnh đi từ đỉnh cao hơn tới đỉnh thấp hơn. Với mọi đỉnh u , tồn tại ít nhất một đường đi từ v_h tới u , và ít nhất một đường đi từ u tới v_l . Các cạnh đi ra từ mỗi đỉnh đã được cho theo thứ tự vị trí của đỉnh kết thúc từ trái sang phải. Yêu cầu dùng một số đường đi từ v_h tới v_l để phủ toàn bộ cạnh của đồ thị, sao cho số đường đi là ít nhất. “Phủ” nghĩa là mỗi cạnh xuất hiện trong ít nhất một đường đi. Các đường đi được chọn có thể chung cạnh với nhau. Trong đề bài và phần phân tích dưới đây, “đường đi” đều là đường đi có hướng; nếu tồn tại đường đi từ a tới b , điều đó không có nghĩa là nhất định tồn tại đường đi từ b tới a .

Đề gốc xem ở phụ lục. Một ví dụ trong tài liệu gốc có đồ thị phẳng cần ít nhất 8 đường đi để phủ hết mọi cạnh.

2.1.1 Mô hình luồng mạng

Phân tích ví dụ một chút, ta rất nhanh liên tưởng tới mô hình luồng mạng cổ điển: đỉnh cao nhất v_h là nguồn của mạng, còn đỉnh thấp nhất v_l là đích. Đường đi trong đề bài chính là dòng chảy từ nguồn tới đích trong mạng. Yêu cầu dùng các đường đi để phủ toàn bộ cạnh và số đường đi ít nhất, tức là yêu cầu lưu lượng trên mỗi cạnh của mạng không nhỏ hơn 1, đồng thời tổng lưu lượng đi ra từ nguồn là nhỏ nhất.

Vì vậy, để giải bài toán chỉ cần xây dựng một mạng có cận dưới dung lượng, rồi tìm luồng khả thi nhỏ nhất của mạng đó. Cách làm đại lược như sau.

Trước hết tìm một luồng khả thi: liệt kê từng cạnh có lưu lượng 0, gọi là (i, j) . Tìm một đường đi từ s tới i và một đường đi từ j tới t . Tăng lưu lượng của mọi cạnh trên đường

$$s \rightarrow i \rightarrow j \rightarrow t$$

lên 1. Như vậy vừa thỏa cân bằng lưu lượng ở mỗi đỉnh, vừa thỏa cận dưới dung lượng của cạnh (i, j) . Sau đó sửa trên luồng khả thi: từ đích về nguồn tìm một luồng ngược khả thi cực đại, ta thu được luồng khả thi nhỏ nhất.

Phân tích độ phức tạp của thuật toán này: khi tìm luồng khả thi, có thể tiền xử lý đường đi từ nguồn và từ mỗi đỉnh tới đích, nên thời gian xây dựng luồng khả thi là $\mathcal{O}(|E| + |V|)$. Khi tìm luồng cực đại, có thể dùng thuật toán tăng đường đơn giản với độ phức tạp $\mathcal{O}(|E|C)$, trong đó C là số lần tăng luồng. Vì đây là đồ thị phẳng nên theo công thức Euler có $\mathcal{O}(|E|) = \mathcal{O}(|V|)$, còn lưu lượng của luồng ngược tối đa là $\mathcal{O}(|E|)$. Do đó độ phức tạp tổng cộng là

$$\mathcal{O}(|V|^2) = \mathcal{O}(n^2).$$

Thuật toán này có hiệu quả thực tế rất cao và có thể nhanh chóng vượt qua dữ liệu kiểm thử. Tuy nhiên, mô hình này chưa khai thác tốt tính chất đồ thị phẳng của đề gốc, nên vẫn còn nhiều khả năng cải tiến.

2.1.2 Mô hình tập thứ tự bộ phận

Đề bài nhấn mạnh rằng mỗi đỉnh đều có hoành độ và tung độ khác nhau, đồ thị là đồ thị phẳng có hướng không chu trình. Hơn nữa, cạnh có hướng giữa hai đỉnh dường như phản ánh một quan hệ so sánh giữa các phần tử. Liệu có mô hình nào tốt hơn để mô tả đồ thị này không? Để bóc lộ bản chất bài toán rõ hơn, ta đưa vào khái niệm tập thứ tự bộ phận.

Các khái niệm liên quan. Một tập thứ tự bộ phận gồm một tập X và một quan hệ hai ngôi R , thỏa các tính chất sau:

- Với mọi $x \in X$, có xRx , tức là R có tính phản xạ.
- Với mọi $x, y \in X$, hễ có xRy và $x \neq y$ thì không có yRx , tức là R có tính phản đối xứng.
- Hễ có xRy và yRz , thì có xRz , tức là R có tính bắc cầu.

Quan hệ thỏa các tính chất ấy gọi là thứ tự bộ phận, thường ký hiệu R bằng $\leq$. $a \leq b$ cũng có thể viết thành $b \geq a$. Nếu có $a \leq b$ và $a \neq b$, ta viết $a < b$ hoặc $b > a$. Quan hệ $<$ còn gọi là quan hệ thứ tự bộ phận nghiêm ngặt. Tập X cùng thứ tự bộ phận $\leq$ được ký hiệu là $(X, \leq)$.

Cho R là một thứ tự bộ phận trên tập X . Với hai phần tử $x, y \in X$, nếu có xRy hoặc yRx , ta nói x và y so sánh được; ngược lại, ta nói chúng không so sánh được. Nếu một quan hệ thứ tự bộ phận R trên X làm cho mọi cặp phần tử của X đều so sánh được, thì R là một thứ tự toàn phần. Ví dụ, quan hệ nhỏ hơn hoặc bằng trên tập số nguyên dương là một thứ tự toàn phần.

Cho a và b là hai phần tử trong tập thứ tự bộ phận $(X, \leq)$. Nếu $a < b$ và trong X không tồn tại phần tử c sao cho $a < c < b$, thì nói a được b phủ, hoặc b phủ a , ký hiệu $a <_c b$. Nếu X là một tập hữu hạn, từ tính bắc cầu của tập thứ tự bộ phận dễ thấy mọi quan hệ thứ tự bộ phận đều có thể được biểu diễn bằng nhiều quan hệ phủ; nói cách khác, ta có thể dùng quan hệ phủ để biểu diễn hữu hiệu quan hệ thứ tự bộ phận.

Biểu diễn đồ thị của tập thứ tự bộ phận hữu hạn $(X, \leq)$, tức biểu đồ Hasse, được mô tả bằng các điểm trên mặt phẳng. Mỗi phần tử của tập thứ tự bộ phận ứng với một điểm. Nếu có $a < b$, thì vị trí của a trên mặt phẳng, nói chính xác là tung độ trong mặt phẳng tọa độ, phải thấp hơn vị trí của b . Nếu $a <_c b$, ta nối một cạnh giữa a và b . Cũng có thể dùng đồ thị có hướng để biểu diễn quan hệ thứ tự bộ phận: mỗi đỉnh ứng với một phần tử; nếu hai phần tử có $a <_c b$, thì trong đồ thị có cạnh có hướng (b, a) từ b tới a . Từ đó có thể thấy đồ thị gốc thực chất là một biểu diễn đồ thị của một tập thứ tự bộ phận.

Chuỗi. Chuỗi là một tập con C của E , trong quan hệ thứ tự bộ phận $\leq$, sao cho mọi cặp phần tử của nó đều so sánh được; tức C là một tập con thứ tự toàn phần của E .

Phản chuỗi. Phản chuỗi, đúng như tên gọi, ngược với chuỗi. Nó là một tập con A của E sao cho trong quan hệ thứ tự bộ phận $\leq$, mọi cặp phần tử của A đều không so sánh được.

Kích thước của chuỗi và phản chuỗi là số phần tử trong tập tương ứng.

Xây dựng mô hình tập thứ tự bộ phận của bài toán gốc. Với các khái niệm ở trên, không khó xây dựng mô hình tập thứ tự bộ phận cho bài toán gốc. Gọi tập thứ tự bộ phận mà đồ thị gốc biểu diễn là $(X, \leq)$, còn tập thứ tự bộ phận mới xây dựng là $(E, \leq)$. Tập E thỏa

$$E = \{(u, v) \mid (u, v) \text{ là một cạnh có hướng trong đồ thị gốc}\},$$

tức các phần tử của E chính là toàn bộ cạnh có hướng trong đồ thị. Cho $a, b \in E$, đặt

$$a = (u_a, v_a), \quad b = (u_b, v_b).$$

Khi và chỉ khi tồn tại một đường đi có hướng từ v_a tới u_b , ta có $a \leq b$, nghĩa là tồn tại một đường đi từ cạnh có hướng a tới cạnh có hướng b . Khi và chỉ khi $v_a = u_b$, ta có $a <_c b$.

Bài toán gốc có thể được phát biểu lại bằng ngôn ngữ tập thứ tự bộ phận: chia tập thứ tự bộ phận $(E, \leq)$ thành ít chuỗi nhất sao cho hợp của các chuỗi này chứa mọi phần tử của E . Tính trực tiếp số chuỗi có vẻ không dễ; may mắn là định lý Dilworth cho thấy quan hệ giữa chuỗi và phản chuỗi, từ đó mục tiêu của bài toán tiếp tục được chuyển hóa.

Định lý 2.1: Định lý Dilworth. Cho $(E, \leq)$ là một tập thứ tự bộ phận hữu hạn. Gọi m là kích thước phản chuỗi lớn nhất trong E , và M là số chuỗi ít nhất để chia E . Khi đó trong E có

$$m = M.$$

Chứng minh xem ở phụ lục.

Theo định lý Dilworth, bài toán chuyển thành tìm kích thước phản chuỗi dài nhất trong E . Nói cách khác, trong đồ thị gốc ta cần chọn càng nhiều cạnh càng tốt, đồng thời bảo đảm các cạnh được chọn đôi một không đi tới nhau được, tức không so sánh được trong $(E, \leq)$. Làm thế nào để tìm phản chuỗi dài nhất? Thật ra điều này liên quan rất lớn tới đồ thị phẳng mà đề gốc cho; tiếp theo ta quay lại đồ thị gốc để thảo luận.

Từ mô hình tập thứ tự bộ phận trở lại đề bài. Vì đồ thị trong đề gốc là đồ thị phẳng, và các đỉnh đi ra cũng được cho từ trái sang phải, nên mọi cạnh trong một phản chuỗi đều có thể được sắp theo thứ tự từ trái sang phải. Nếu dùng một đường nối các cạnh ứng với phản chuỗi dài nhất từ trái sang phải, như trong hình dưới, đường này sẽ không cắt các cạnh khác của đồ thị phẳng. Nói chính xác hơn:

Định lý 2.2. Sắp các cạnh ứng với phản chuỗi dài nhất từ trái sang phải; hai cạnh kề nhau trong thứ tự ấy nhất định nằm trong cùng một miền, tức cùng một mặt đóng. Miền ở đây là phần mặt được bao bởi hai đường đi khác nhau từ một đỉnh tới một đỉnh khác, một đỉnh là cực cao, một đỉnh là cực thấp; hai đường đi không có cạnh chung. Bên trong mặt này không có đỉnh và cạnh nào khác; ký hiệu miền là F . Trong hai đường đi bao quanh miền F , đường bên trái được định nghĩa là biên trái của F , còn đường bên phải được định nghĩa là biên phải của F .

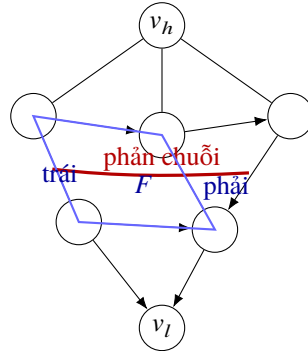


Figure 1: Một phản chuỗi trong đồ thị phẳng có thể nối từ trái sang phải mà không cắt cạnh khác; hai cạnh kề trong thứ tự ấy nằm trên hai biên của cùng một miền F .

Chứng minh ngắn gọn của Định lý 2.2 xem ở phụ lục.

Gợi ý từ Định lý 2.2 cho phép dùng truy hồi để tìm độ dài phản chuỗi dài nhất trong đồ thị. Đặt $f(x)$ là độ dài phản chuỗi dài nhất trong vùng phẳng nằm bên trái cạnh x và kết thúc bằng x . Giả sử x nằm trên biên phải của một miền F . Khi đó

$$f(x) = 1 + \max\{f(y) \mid y \text{ nằm trên biên trái của } F\}.$$

Vì theo Định lý 2.2, nếu x nằm trong một phản chuỗi dài nhất, thì cạnh kề với x trong phản chuỗi và nằm bên trái x chỉ có thể nằm trên biên trái của miền F . Sau khi có công thức truy hồi này, chỉ cần tìm từng miền theo thứ tự từ trái sang phải, từ trên xuống dưới, rồi thực hiện truy hồi.

2.1.3 Thiết kế thuật toán tương ứng

Có thể dùng DFS để tìm các miền trong đồ thị phẳng. Trong DFS cần ghi hai thông tin: màu của đỉnh và đỉnh cha đã mở rộng nó. Màu của mỗi đỉnh được ghi bằng $C[u]$. $C[u]$ có ba trạng thái: trắng nghĩa là đỉnh u chưa được duyệt, ban đầu mọi đỉnh đều trắng; xám nghĩa là đỉnh u đã được duyệt nhưng chưa kiểm tra xong, tức nó vẫn còn đỉnh kế tiếp chưa mở rộng; đen nghĩa là đỉnh u đã được duyệt và cũng đã kiểm tra xong. Đỉnh cha mở rộng nó được ghi bằng pre .

Khung thuật toán đại lược:

```
DFS(dỉnh u)
begin
  C[u] <- xám
  while v là đỉnh kế tiếp của u do
    (mở rộng các đỉnh kế tiếp v của u từ trái sang phải)
    if C[v] là trắng
      then begin
        pre[v] <- u
        DFS(v)
      end else begin
        vl <- v
        vh <- v
        while C[vh] là đen do
          vh <- pre[vh]
          (đen nghĩa là đỉnh trên biên của miền;
           xám nghĩa là đỉnh cực cao)
        tính truy hồi f(x) trên biên phải
          (các cạnh truy vết theo pre là biên trái;
           đương tu vh tới vl là biên phải)
        pre[v] <- u
      end
    end
  C[u] <- đen
end
```

Tính độ phức tạp của thuật toán trên. Thứ nhất, DFS trên mỗi đỉnh có độ phức tạp $\mathcal{O}(|E|)$. Thứ hai, quay lui để tìm các cạnh trên biên của mỗi miền và thực hiện truy hồi. Vì đây là đồ thị phẳng, mỗi cạnh nhiều nhất thuộc biên của hai miền khác nhau, nên bước này có độ phức tạp $\mathcal{O}(|E| + |F|)$. Do đồ thị trong đề gốc là đồ thị phẳng, theo định lý Euler, số cạnh $|E|$ và số miền, hay số mặt, $|F|$ đều ở cấp $\mathcal{O}(n)$. Vì vậy độ phức tạp thời gian tổng cộng là $\mathcal{O}(n)$.

2.1.4 Tiểu kết

Phương pháp thứ nhất dùng mô hình luồng mạng, trực tiếp thể hiện tính chất mạng có hướng không chu trình của đề gốc. Cách giải có tính tổng quát, nhưng chưa thể hiện đầy đủ tính chất đồ thị phẳng của đề. Phương pháp thứ hai dùng mô hình tập thứ tự bộ phận để chuyển hóa mục tiêu bài toán, đưa bài toán từ luồng mạng trở lại bản thân đồ thị phẳng, qua đó bộc lộ trọn vẹn bản chất của bài toán. Chính vì đã quay về bản chất, ở phía sau ta mới có thể dùng DFS để khai thác đầy đủ tính chất đồ thị phẳng và thu được thuật toán có độ phức tạp tối ưu.

Từ sự so sánh hai phương pháp trên có thể thấy hai mô hình đồ thị khác nhau dẫn tới khác biệt lớn về độ phức tạp thời gian của thuật toán. Điều đó cho thấy tầm quan trọng của việc chọn mô hình. Như câu

nói “mài dao không làm lờ việc đốn củi”, trước khi thiết kế thuật toán, chọn một mô hình đồ thị đúng thường có thể đem lại hiệu quả gấp bội: không chỉ giảm độ khó của thiết kế thuật toán, mà còn làm cho thuật toán được thiết kế đơn giản và hiệu quả.

Khi chọn mô hình đồ thị thích hợp cho một bài toán thực tế, không nên chỉ bó hẹp ở vài tính chất biểu hiện trên bề mặt bài toán rồi giải theo kinh nghiệm sẵn có; làm vậy dễ rơi vào lối mòn và không đạt thuật toán hiệu quả nhất. Bài mới nên làm theo cách mới: cần đổi mới cách nghĩ, phân tích sâu các tính chất khác nhau của bài toán, kết hợp các tính chất ấy lại để tìm ra mô hình đồ thị thể hiện rõ nhất bản chất bài toán.

Nhiều khi thuật toán cuối cùng không được thiết kế trực tiếp trên mô hình đã chọn. Như phương pháp thứ hai, tập thứ tự bộ phận không xuất hiện trong DFS, nhưng một khi nghĩ tới tập thứ tự bộ phận thì có thể nói bài toán đã giải được một nửa. Mô hình đồ thị nhiều hơn là một bước chuyển trong suy nghĩ, làm cho mạch tư duy trở nên rõ ràng, như một ngọn đèn chỉ đường dẫn ta tới bờ thành công.

3 Khai thác và tận dụng đầy đủ tính chất của mô hình

Phần trước nói về tầm quan trọng của việc chọn mô hình thích hợp và phương pháp xây dựng mô hình đồ thị. Việc lập mô hình giống như dựng một nền tảng cho thiết kế thuật toán; công việc tiếp theo là trên nền tảng ấy khai thác và tận dụng đầy đủ các tính chất của đề gốc để thiết kế một thuật toán tốt. Khai thác và tận dụng tính chất của mô hình như thế nào? Dưới đây ta lại dùng một ví dụ cụ thể để giải thích.

3.1 Ví dụ 2: Alice và Bob

Nguồn bài. ACM Central European Programming Contest 2001, Problem A: *Alice and Bob*.

Mô tả đề bài. Alice và Bob đang chơi một trò chơi. Alice vẽ một đa giác lồi có n đỉnh, $n \leq 10000$, rồi gán nhãn các đỉnh của đa giác từ 1 đến n theo một thứ tự tùy ý. Sau đó cô vẽ một số đường chéo không cắt nhau trong đa giác; nếu hai đường chỉ chung đỉnh thì không tính là cắt nhau. Cô cho Bob biết hai đầu mút của mỗi cạnh và mỗi đường chéo, nhưng không cho biết đường nào là cạnh, đường nào là đường chéo. Bob phải đoán ra thứ tự nhãn các đỉnh theo chiều kim đồng hồ hoặc ngược chiều kim đồng hồ; chỉ cần một dãy phù hợp bất kỳ.

Đề gốc xem ở phụ lục. Ví dụ, $n = 5$, các cạnh hoặc đường chéo được cho là

$$(1, 4), (4, 2), (1, 2), (5, 1), (2, 5), (5, 3), (1, 3).$$

Một thứ tự nhãn đỉnh có thể là

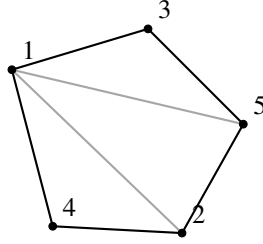
$$(1, 3, 5, 2, 4).$$

Đa giác tương ứng được minh họa trong hình dưới.

3.1.1 Xây dựng mô hình

Theo ý nghĩa đề bài, không khó nhận ra mô hình đồ thị của nó: đa giác lồi ứng với một đồ thị G có n đỉnh; các cạnh của đa giác và các đường chéo ứng với cạnh trong G . Hơn nữa, rất rõ ràng G là đồ thị phẳng; theo công thức Euler, số cạnh trong đồ thị ở cấp $\mathcal{O}(n)$.

Nghiên cứu tính chất đồ thị phẳng của G , ta thu được Kết luận 3.1: một đường chéo chia đa giác lồi thành hai phần, mỗi phần đều chứa ít nhất một đỉnh khác đầu mút của đường chéo, và hai phần ấy nằm ở hai phía của đường chéo. Từ đó biết rằng trong G chỉ tồn tại duy nhất một chu trình Hamilton. Vì đường chéo sẽ chia đa giác thành hai phần không còn liên thông với nhau qua biên, nên đường chéo không thể



Chu trình ngoài là thứ tự (1, 3, 5, 2, 4);
các dây xám là đường chéo được trộn với cạnh trong dữ liệu vào.

Figure 2: Mô hình của ví dụ Alice và Bob: từ tập cạnh và đường chéo không cắt nhau, cần khôi phục chu trình ngoài của đa giác.

nằm trên chu trình Hamilton. Do đó các cạnh trên chu trình Hamilton đều là cạnh của đa giác. Cạnh của đa giác chỉ có n cạnh, nên chu trình Hamilton cũng chỉ có một. Gọi chu trình Hamilton này là

$$H_1, H_2, H_3, \dots, H_n.$$

Mục tiêu của bài toán là tìm chu trình Hamilton đó. Dưới đây thảo luận cách tận dụng các tính chất này để thiết kế thuật toán.

3.1.2 Dùng tính liên thông của cạnh

Theo Kết luận 3.1, nếu một cạnh là đường chéo thì sau khi xóa hai đầu mút của đường chéo khỏi G , đồ thị G nhất định trở thành hai thành phần liên thông không đi tới nhau được. Ngược lại, nếu một cạnh là cạnh của đa giác, thì sau khi xóa hai đầu mút của cạnh ấy khỏi G , đồ thị G vẫn liên thông. Nói cách khác, có thể dựa vào tính liên thông của cạnh trong G để phán đoán một cạnh là đường chéo hay là cạnh của đa giác. Từ đó có thuật toán thứ nhất:

Thuật toán 1.

1. Liệt kê từng cạnh (u, v) trong đồ thị và phán đoán như sau: xóa hai đỉnh u và v khỏi G , thu được đồ thị mới G' . Chọn tùy ý một đỉnh a trong G' , rồi duyệt G' bắt đầu từ a . Nếu a tới được mọi đỉnh khác trong G' , thì G' liên thông và (u, v) là cạnh của đa giác; ngược lại, G' không liên thông và (u, v) là đường chéo.
2. Xóa mọi đường chéo khỏi G , thu được đồ thị mới C . Khi đó C chính là chu trình Hamilton trong G , nên chỉ cần duyệt nó một lần là thu được thứ tự nhãn các đỉnh của đa giác.

Phân tích độ phức tạp của Thuật toán 1: trong bước 1, số cạnh cần liệt kê nhiều nhất là $2n$; mỗi lần kiểm tra tính liên thông của đồ thị tốn $\mathcal{O}(n)$, nên độ phức tạp là $\mathcal{O}(n^2)$. Trong bước 2, xóa cạnh và duyệt đồ thị mới đều tốn $\mathcal{O}(n)$. Vì vậy độ phức tạp tổng cộng là $\mathcal{O}(n^2)$. Khi n có thể lên tới 10000, độ phức tạp của Thuật toán 1 vẫn hơi cao, nên cần tiếp tục tối ưu.

3.1.3 Chia để trị

Tiếp tục khai thác tính chất của G , ta thấy do các đường chéo trong G không cắt nhau, nhất định tồn tại một đỉnh u có bậc bằng 2. Hơn nữa, có thể khẳng định hai cạnh nối với u đều là cạnh của đa giác. Xóa tam giác có đỉnh u khỏi đa giác thì phần còn lại vẫn là một đa giác. Có thể dùng tính chất này để phán đoán một cạnh là cạnh của đa giác hay đường chéo không? Câu trả lời là có.

Vì trong đồ thị nhất định tồn tại một đỉnh u có bậc bằng 2. Giả sử u là đỉnh H_i trên chu trình Hamilton, hai đỉnh kề của nó là H_{i-1} và H_{i+1} . Hai cạnh (H_{i-1}, H_i) và (H_i, H_{i+1}) đều là cạnh của đa giác; thêm hai cạnh ấy vào một đồ thị phụ C . Ban đầu đồ thị phụ C chỉ có n đỉnh ứng với các đỉnh đa giác và chưa có cạnh. Sau khi cắt tam giác có đỉnh u , tức tam giác $H_i H_{i-1} H_{i+1}$, khỏi đa giác, hình còn lại vẫn là một đa giác. Nghĩa là ta có thể xóa H_i khỏi G ; nếu giữa H_{i-1} và H_{i+1} chưa có cạnh thì thêm một cạnh mới (H_{i-1}, H_{i+1}) , gọi là cạnh ảo, để thu được đồ thị mới G' . Khi đó trong G' vẫn tồn tại duy nhất một chu trình Hamilton

$$\dots, H_{i-2}, H_{i-1}, H_{i+1}, H_{i+2}, \dots$$

Đồ thị G' có cùng tính chất với G , vì vậy cũng tồn tại ít nhất một đỉnh bậc 2, gọi là H_j . Khi đó hai cạnh (H_{j-1}, H_j) và (H_j, H_{j+1}) có thể là cạnh của đa giác, đường chéo của đa giác hoặc cạnh ảo vừa thêm; ta thêm vào C những cạnh nào trong đó là cạnh của đa giác. Sau đó lại xóa H_j khỏi G' theo cùng cách, thu được một đồ thị mới khác. Cứ như vậy, liên tục tìm đỉnh bậc 2 trong đồ thị mới, thêm vào C những cạnh tương ứng là cạnh đa giác, rồi xóa đỉnh ấy khỏi đồ thị. Lặp lại cho tới khi đồ thị không còn đỉnh bậc 2. Sau đó, trong các cạnh còn lại của đồ thị, thêm vào C những cạnh là cạnh của đa giác. Cuối cùng, C chính là đa giác gốc cần tìm.

Quá trình mô phỏng có thể nhìn như trong hình dưới: bên trái là sự thay đổi của đồ thị G , còn bên phải là sự thay đổi của đồ thị C .

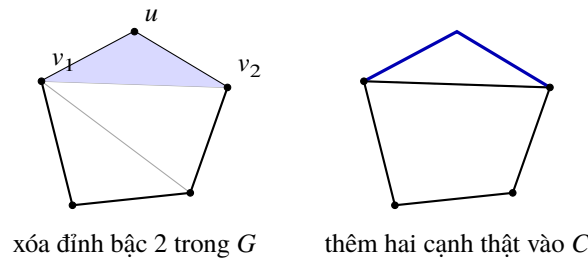


Figure 3: Bước chia để trị: khi u có bậc 2, hai cạnh kề u là cạnh đa giác, còn cạnh nối hai láng giềng trở thành cạnh ảo cho đa giác còn lại.

Còn một vấn đề chưa giải quyết: làm thế nào để phán đoán cạnh bị loại bỏ có phải là cạnh của đa giác gốc hay không? Trước hết xét khi nào một cạnh (H_i, H_j) mới bị loại bỏ. Một cạnh (H_i, H_j) khi và chỉ khi trở thành cạnh của một đa giác nào đó mới có khả năng bị loại bỏ. Nếu (H_i, H_j) là một đường chéo của đa giác gốc, thì khi và chỉ khi mọi đỉnh trên chu trình Hamilton từ H_{i+1} tới H_{j-1} đều đã bị xóa, cạnh (H_i, H_j) mới có thể trở thành cạnh của đa giác mới. Làm thế nào để phán đoán các đỉnh từ H_{i+1} tới H_{j-1} đã bị xóa hay chưa? Khi ấy đồ thị phụ C vừa xây dựng phát huy tác dụng. Nếu các đỉnh từ H_{i+1} tới H_{j-1} đã bị xóa, thì các cạnh tương ứng của chúng trên đa giác gốc đều đã được thêm vào C , và trong C nhất định tồn tại một đường đi từ H_i tới H_j . Vì vậy, để phán đoán một cạnh bị loại bỏ (H_i, H_j) có phải đường chéo hay không, chỉ cần phán đoán H_i và H_j có liên thông trong C hay không. Cần chú ý rằng khi C đã có $n - 1$ cạnh, cạnh còn lại sẽ bị phán đoán thành đường chéo, nhưng lúc này đã có thể xác định dãy nhãn đỉnh của đa giác.

Khung thuật toán đã rõ. Tiếp theo chọn cấu trúc dữ liệu phù hợp cho thuật toán:

Lưu đồ thị G . Vì G là đồ thị thưa, có thể dùng danh sách kề để lưu, phục vụ việc tìm đỉnh bậc 2 nối với những cạnh nào. Ngoài ra, cần dùng một bảng băm để lưu mọi cạnh, phục vụ việc kiểm tra hai đỉnh bất kỳ có cạnh nối trực tiếp hay không. Với một cạnh (u, v) , bảng băm có thể dùng

$$un + v$$

làm khóa, và dùng hàm băm

$$h(u, v) = (un + v) \bmod P,$$

trong đó P là một số nguyên tố lớn. Nhờ vậy, trong khi vẫn bảo đảm độ phức tạp tra cứu là $\mathcal{O}(1)$, không gian lưu trữ nhỏ hơn ma trận kề rất nhiều.

Liệt kê đỉnh bậc 2. Rất dễ nghĩ tới việc dùng heap nhỏ nhất để tìm đỉnh bậc 2, nhưng độ phức tạp như vậy hơi cao. Vì bậc của đỉnh chỉ giảm chứ không tăng, và bậc thật sự có ích chỉ là 1 và 2, ta có thể xây dựng bốn thùng thay cho heap. Đặt đỉnh vào bốn thùng khác nhau theo bậc: bậc 0, bậc 1, bậc 2, và bậc lớn hơn 2. Trong mỗi thùng, dùng danh sách liên kết đôi để lưu các đỉnh khác nhau. Khi bậc của một đỉnh bị sửa, xóa nó khỏi thùng cũ rồi đưa vào thùng tương ứng. Chèn và xóa đỉnh trong danh sách liên kết đôi đều tốn $\mathcal{O}(1)$. Vì bậc của mỗi đỉnh liên tục giảm, mỗi đỉnh nhiều nhất vào ra mỗi thùng một lần. Tóm lại, thời gian cần để tìm và điều chỉnh một đỉnh bậc 2 trong các thùng là $\mathcal{O}(1)$.

Thêm cạnh vào C , phán đoán liên thông giữa hai đỉnh bất kỳ. Có thể dùng cấu trúc hợp nhất - tìm kiếm để thực hiện việc thêm cạnh, tức gộp hai đồ thị con liên thông, và phán đoán tính liên thông của hai đỉnh, tức kiểm tra chúng có nằm trong cùng một đồ thị con liên thông hay không. Cả hai thao tác thêm một cạnh và kiểm tra liên thông của một cặp đỉnh đều có độ phức tạp khấu hao $\mathcal{O}(\alpha(n))$.

Thuật toán 2.

1. Theo bậc của mỗi đỉnh, đưa n đỉnh vào bốn thùng. Khởi tạo đồ thị phụ C .
2. Nếu tồn tại một đỉnh u có bậc 2, thực hiện bước 3; ngược lại chuyển tới bước 6.
3. Trong danh sách kề, tìm hai đỉnh nối với u , gọi là v_1 và v_2 .
4. Trước hết phán đoán (u, v_1) và (u, v_2) có phải cạnh ảo hay không. Nếu không phải cạnh ảo, dùng hợp nhất - tìm kiếm kiểm tra chúng có phải đường chéo hay không. Cuối cùng, chèn các cạnh là cạnh của đa giác vào C .
5. Đánh dấu đỉnh u là không còn dùng được. Dùng bảng băm kiểm tra cạnh (v_1, v_2) có tồn tại trong G hay không. Nếu (v_1, v_2) không tồn tại, thêm một cạnh ảo (v_1, v_2) , bậc của v_1 và v_2 không đổi; ngược lại giảm bậc của v_1 và v_2 đi 1. Nếu bậc mới của v_1 hoặc v_2 không phù hợp với thùng đang chứa nó, điều chỉnh thùng. Sau đó quay lại bước 2.
6. Tìm mọi đỉnh bậc 1 trong đồ thị còn lại G , lần lượt kiểm tra tính liên thông trong đồ thị phụ C của cạnh tương ứng với các đỉnh này, rồi chèn vào C các cạnh không phải đường chéo và các cạnh ảo.
7. Duyệt C để thu được thứ tự nhãn đỉnh của đa giác gốc.

Khi mỗi đỉnh bị xóa, nhiều nhất một cạnh ảo được thêm vào, nên tổng số cạnh gốc và cạnh ảo vẫn ở cấp $\mathcal{O}(n)$. Bây giờ phân tích độ phức tạp thời gian: bước 3, mỗi đỉnh nhiều nhất được tìm một lần, nên độ phức tạp bằng số cạnh, tức $\mathcal{O}(n)$; bước 4 và bước 6, dùng hợp nhất - tìm kiếm để tìm và chèn có thời gian khấu hao $\mathcal{O}(n\alpha(n))$; bước 5, dùng bảng băm tra một cạnh tốn $\mathcal{O}(1)$, chèn cạnh và điều chỉnh thùng cũng là $\mathcal{O}(1)$, tổng cộng thực hiện số lần bằng số đỉnh, nên thời gian là $\mathcal{O}(n)$. Tóm lại, độ phức tạp của Thuật toán 2 là $\mathcal{O}(n\alpha(n))$.

Độ phức tạp thời gian của Thuật toán 2 đã rất thấp. Nhưng khi theo đuổi không ngừng, ta không thể dừng lại ở mức vừa chạm tới, vì Thuật toán 2 vẫn còn điểm cần cải tiến: độ phức tạp lập trình quá cao. Thuật toán 2 tuy đã khai thác khá đầy đủ các tính chất của bài toán, nhưng khi sử dụng các tính chất ấy, nó thực hiện bằng cách liên tục tách bài toán thành các bài toán con rồi lần lượt phá vỡ từng bài toán. Ta không khỏi đặt câu hỏi: liệu có tìm được một phương pháp tổng hợp, sử dụng mọi tính chất cùng lúc, sao cho lời giải vừa đơn giản vừa hiệu quả hay không?

3.1.4 Cây DFS phản ánh tính chất bài toán

Quay lại đồ thị phẳng nơi đa giác nằm. Nếu duyệt đồ thị phẳng theo chiều sâu, khi đi qua một đường chéo, đồ thị phẳng bị chia thành hai phần. Vì là duyệt theo chiều sâu, trong các đỉnh còn lại của hai phần này, sẽ có một phần được duyệt trước. Do đó ta có thể mô phỏng thuật toán DFS tìm khối, dùng thứ tự duyệt đỉnh và hàm low để phán đoán tính chất của cạnh.

Trước hết định nghĩa hai hàm cần dùng trong DFS:

$dfn[v]$ là thứ tự mà v được duyệt trong tìm kiếm sâu,

$$low[v] = \min(dfn[v], \min_{w_1 \text{ là con của } v} low[w_1], \min_{w_2 \text{ là tổ tiên kề với } v, w_2 \neq parent(v)} dfn[w_2]),$$

trong đó w_1 biểu thị đỉnh con của v , còn w_2 biểu thị các đỉnh tổ tiên khác cha của v trong cây DFS.

Tiếp theo xét từng trường hợp để xem dùng dfn và low phán đoán tính chất cạnh như thế nào.

Xét một cạnh (u, v) trên cây DFS, trong đó u là cha của v . Vì G là một khối, số con của mỗi đỉnh không vượt quá 2. Theo số con của v , ta chia bốn trường hợp sau:

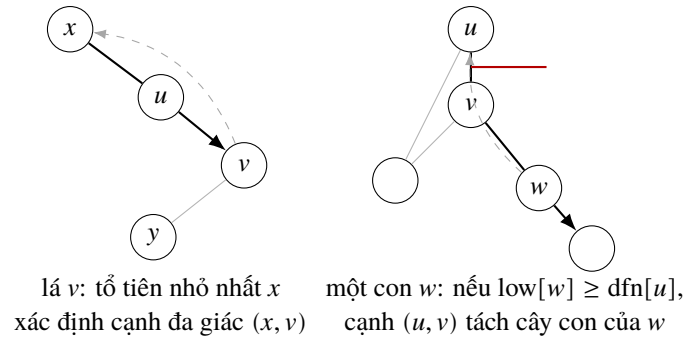


Figure 4: Hai cấu hình DFS dùng trong các trường hợp chính: lá v và đỉnh v có một con w .

1. v có 0 con: điều này nghĩa là (u, v) là cạnh của đa giác gốc. Nếu không, mọi đỉnh tổ tiên của v đều ở một phía của (u, v) , còn phía kia chắc chắn vẫn tồn tại đỉnh, mâu thuẫn với việc v không có con. Gọi x là đỉnh có giá trị dfn nhỏ nhất trong các tổ tiên nối trực tiếp với v . Khi đó (x, v) nhất định cũng là cạnh của đa giác gốc. Lược chứng: như cấu hình bên trái trong hình trên, giả sử (x, v) là cạnh của đa giác. v không phải cha của x , nên x nhất định được duyệt trước u , và x là tổ tiên của u . Nói cách khác, tồn tại một đường đi từ x tới u , và các cạnh trên đường đi đều là cạnh của cây DFS. Gọi y là đỉnh khác x và u nhưng nối với v . Vì G là đồ thị phẳng, y nhất định nằm trên đường từ x tới u . Do đó y nhất định được duyệt sau x , tức $dfn[y]$ lớn hơn $dfn[x]$. Vì vậy x chắc chắn là đỉnh có dfn nhỏ nhất trong các tổ tiên nối với v .
2. v có 1 con và u là gốc của cây DFS: dễ thấy (u, v) nhất định là cạnh của đa giác.
3. v có 1 con và u không phải gốc của cây DFS: gọi w là con duy nhất của v . Nếu $low[w] \geq dfn[u]$, thì (u, v) là đường chéo của đa giác. Vì (u, v) chia mặt phẳng thành hai phần, w và các đỉnh trong cây con của nó không thể duyệt tới phần kia, như cấu hình bên phải trong hình trên. Gọi x là đỉnh có dfn nhỏ nhất trong các tổ tiên nối trực tiếp với v . Khi đó (x, v) nhất định là cạnh của đa giác gốc, chứng minh giống phân tích ở trường hợp 1. Cạnh đa giác còn lại nối với v có thể được tìm khi duyệt w . Nếu $low[w] < dfn[u]$, thì (u, v) là cạnh của đa giác, còn cạnh đa giác còn lại nối với v có thể được tìm khi duyệt w .
4. v có 2 con: hai cạnh đa giác nối với v có thể được tìm khi duyệt các đỉnh con.

Xét bốn trường hợp trên là đủ để phán đoán tính chất của mọi cạnh trong G . Vì vậy chỉ cần DFS một lần trên G , ta có thể xác định thứ tự nhân đỉnh của đa giác. Độ phức tạp thời gian của Thuật toán 3 vì thế là tuyến tính:

$$\mathcal{O}(n).$$

3.1.5 Tiểu kết

Ba phương pháp trên đều được xây dựng trên cùng một mô hình; mức độ khai thác tính chất mô hình khác nhau quyết định độ phức tạp thời gian khác nhau. Thuật toán 1 dùng phương pháp định nghĩa, dựa vào tính chất đường chéo chia đa giác thành hai phần để tìm mọi đường chéo của đa giác. Trong quá trình thiết kế Thuật toán 2, ta phát hiện rằng sau mỗi lần xóa một góc của đa giác, phần còn lại vẫn là đa giác, tức đã tìm ra tính tương tự của bài toán con; nhờ đó có thể liên tục thu nhỏ quy mô bài toán. Thuật toán 2 còn thể hiện tư tưởng quy về bài quen: chia bài toán chưa biết thành vài bước, hoặc vài bài toán con, mà mỗi bước đều quen thuộc với ta, từ đó có thể chọn thuật toán tốt nhất cho từng bước. Vì mỗi bài toán con đều được giải, bài toán gốc cũng được giải quyết dễ dàng. Có thể nói phương pháp thứ hai là một phương pháp phân tích. Thuật toán 3 thể hiện tư tưởng tổng hợp: không xét riêng lẻ tính liên thông của từng cạnh, mà xét từ toàn cục, phát hiện sự khác biệt giữa cạnh đa giác và đường chéo trong cây DFS. Vì vậy thiết kế Thuật toán 3 liền mạch, thể hiện vẻ đẹp của một loại “thứ tự” trong lý thuyết đồ thị.

Ba phương pháp khác nhau cũng đem lại cho ta các gợi ý khác nhau. Phương pháp định nghĩa nói với ta rằng khi không tìm được hướng suy nghĩ cho bài toán, có thể xét từ những tính chất cơ bản nhất của bài toán để tìm điểm đột phá. Lợi ích của phương pháp phân tích là các bài toán con sau khi phân rã thường đơn giản hơn bài toán gốc; điểm khó là mỗi phần đều ảnh hưởng tới độ phức tạp cuối cùng của thuật toán, vì vậy thiết kế thuật toán cho từng phần phải thật tinh tế. Việc vận dụng phương pháp tổng hợp đòi hỏi cái nhìn toàn cục và khả năng phát hiện đặc tính mô hình tiêu biểu nhất.

4 Tổng kết

Từ “mô hình” từng được nhắc tới trong vô số bài viết. Nó là tinh hoa của tư tưởng cơ bản trong lý thuyết đồ thị, là mấu chốt để giải các bài toán đồ thị. Xây dựng mô hình yêu cầu ta quen thuộc với các mô hình cổ điển, đồng thời cũng yêu cầu ta dám tìm tòi, mạnh dạn sáng tạo, dám nhảy ra khỏi khuôn khổ mô hình cổ điển. Tận dụng tính chất của mô hình đòi hỏi ta có con mắt tinh tường, nắm được bản chất bài toán và đánh trúng điểm then chốt.

Đồng thời bài viết cũng giới thiệu ba phương pháp giải bài toán: phương pháp định nghĩa, phương pháp phân tích và phương pháp tổng hợp. Cũng như “mô hình”, các phương pháp này có tính phổ quát trong việc giải các bài toán thi Olympic Tin học. Chúng không chỉ là phương pháp, mà còn là cách tư duy và góc độ suy nghĩ. Vận dụng và nắm vững chúng một cách linh hoạt sẽ có lợi cho việc nghiên cứu thuật toán và khám phá khoa học.

Quá trình phân tích ví dụ thứ hai không chỉ thể hiện tư tưởng cơ bản của lý thuyết đồ thị, mà còn cho thấy sự kết hợp hoàn mỹ giữa thuật toán và cấu trúc dữ liệu, cũng như tư tưởng tối ưu hóa thuật toán. Sự kết hợp giữa thuật toán và cấu trúc dữ liệu là yêu cầu tất yếu của lập trình xuất sắc, là biểu hiện của việc dung hội kiến thức. Tối ưu hóa nhằm hoàn thiện hơn nữa thiết kế chương trình, là tinh thần không ngừng tiến lên. Những điều này cũng là tố chất cần thiết cho nghiên cứu khoa học sau này.

Bài viết này có lẽ chưa tổng kết hoàn mỹ các tư tưởng và phương pháp cơ bản của lý thuyết đồ thị; nó chỉ là một chút hiểu biết và suy nghĩ của tác giả về lý thuyết đồ thị. Nhưng tác giả hy vọng bài viết có thể gợi mở cho người đọc, giúp người đọc suy nghĩ sâu và tổng kết các thuật toán cùng phương pháp cơ bản của lý thuyết đồ thị. Dù biến hóa muôn dạng, mọi thứ vẫn không rời gốc; chỉ những tư tưởng và phương pháp cơ bản nhất mới là con đường đúng đắn để giải bài toán.

Tài liệu tham khảo

- [1] Richard A. Brualdi, *Introductory Combinatorics*, 3rd edition.
- [2] Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi Tin học*.
- [3] Poland Olympiad of Informatics 2002 Stage III: *Skiers*.
- [4] ACM Central European Programming Contest 2001 Problems and Solutions.

Lời cảm ơn

Chân thành cảm ơn thầy Xiang Qizhong đã hướng dẫn và giúp đỡ tôi trong học tập.

Chân thành cảm ơn bạn Li Shi đã gợi ý cho tôi trong ví dụ thứ nhất.

Chân thành cảm ơn Hu Weidong và Wang Jun đã giúp đỡ tôi rất nhiều.

Thành thật cảm ơn hai bạn Zhou Gelin và Xiao Xiangning đã đưa ra những ý kiến quý báu cho bài viết của tôi.

A Phụ lục

A.1 Chứng minh định lý Dilworth

Dễ thấy M nhất định không nhỏ hơn m , vì các phần tử trong phản chuỗi dài nhất nhất định phải nằm trong các chuỗi khác nhau. Vì vậy chỉ cần chứng minh chắc chắn tồn tại cách chia với $M \leq m$. Tiếp theo, ta chứng minh bằng quy nạp theo số phần tử n của E .

Khi $n = 1$, định lý hiển nhiên đúng.

Khi $n > 1$, chia hai trường hợp.

Trường hợp 1. Tồn tại một phản chuỗi kích thước m ,

$$A = \{a_1, a_2, \dots, a_m\},$$

không phải là tập toàn bộ phần tử cực đại của P , cũng không phải là tập toàn bộ phần tử cực tiểu của P .

Định nghĩa

$$A^- = \{x \in P \mid \exists a_i \in A, x \leq a_i\},$$

tức với mọi phần tử x trong A^- , có thể tìm một phần tử a_i trong A sao cho $x \leq a_i$. Tương tự, định nghĩa

$$A^+ = \{x \in P \mid \exists a_i \in A, x \geq a_i\}.$$

Trực quan mà nói, A^- gồm mọi phần tử nằm “phía dưới” A , còn A^+ gồm mọi phần tử nằm “phía trên” A . Hơn nữa các tính chất sau đều đúng:

1. Vì tồn tại ít nhất một phần tử cực đại không thuộc A , phần tử cực đại ấy cũng không thuộc A^- . Do đó $A^- \neq P$, và số phần tử của A^- nhỏ hơn số phần tử n của P .
2. Vì tồn tại ít nhất một phần tử cực tiểu không thuộc A , phần tử cực tiểu ấy cũng không thuộc A^+ . Do đó $A^+ \neq P$, và số phần tử của A^+ nhỏ hơn n .
3. $A^- \cap A^+ = A$. Phản chứng: giả sử tồn tại một x thuộc $A^- \cap A^+$ nhưng không thuộc A . Theo giả thiết phản chứng, trong A tồn tại hai phần tử a_i và a_j sao cho $a_i \leq x \leq a_j$, mâu thuẫn với việc A là một phản chuỗi. Vì vậy tính chất 3 đúng.

4. $A^- \cup A^+ = P$. Phản chứng: giả sử tồn tại một x không thuộc $A^- \cup A^+$ nhưng thuộc P . Theo giả thiết phản chứng, với mọi $a_i \in A$, đều không thỏa $a_i \leq x$ và cũng không thỏa $a_i \geq x$. Nói cách khác, x không so sánh được với mọi phần tử trong A , vậy $A \cup \{x\}$ là một phản chuỗi dài hơn A , mâu thuẫn với việc A là phản chuỗi dài nhất trong P . Vì vậy tính chất 4 đúng.

Vì số phần tử của A^+ và A^- đều nhỏ hơn n , theo giả thiết quy nạp, A^+ và A^- đều có thể được chia thành m chuỗi. Giả sử A^+ được chia thành

$$C_1^+, C_2^+, \dots, C_m^+,$$

và $a_i \in C_i^+$. Tương tự, giả sử A^- được chia thành

$$C_1^-, C_2^-, \dots, C_m^-,$$

và $a_i \in C_i^-$. Rõ ràng, với mọi i , a_i là phần tử cực tiểu của C_i^+ . Vì nếu phần tử cực tiểu trong C_i^+ không phải a_i mà là x , thì theo định nghĩa của A^+ , tồn tại một $a_j \leq x$, suy ra $a_j \leq x \leq a_i$, mâu thuẫn với việc A là phản chuỗi. Tương tự, a_i là phần tử cực đại của C_i^- . a_i là phần tử kết thúc của C_i^- và là phần tử bắt đầu của C_i^+ ; nối C_i^- với C_i^+ tạo thành m chuỗi, và các chuỗi này là một phép chia của P .

Trường hợp 2. Tối đa chỉ tồn tại hai phản chuỗi kích thước m : một hoặc cả hai trong tập toàn bộ phần tử cực đại và tập toàn bộ phần tử cực tiểu. Gọi x là một phần tử cực tiểu, y là một phần tử cực đại và $x \leq y$, trong đó x có thể bằng y . Khi đó phản chuỗi lớn nhất của

$$P - \{x, y\}$$

có kích thước $m - 1$. Theo giả thiết quy nạp, $P - \{x, y\}$ có thể được chia thành $m - 1$ chuỗi. Các chuỗi này cùng với chuỗi $\{x, y\}$ tạo thành một phép chia của P .

A.2 Chứng minh Định lý 2.2

Phản chứng: giả sử trong phản chuỗi dài nhất A tồn tại hai cạnh kề nhau không nằm trong cùng một miền. Gọi hai cạnh đó là a và b , với a nằm bên trái b . Một cạnh nhiều nhất thuộc hai miền khác nhau. Có ba tình huống:

1. a là một cạnh trên biên trái của một miền F_1 . Khi đó tồn tại một cạnh c trên biên phải của F_1 không so sánh được với cả a và b . Vậy $A \cup \{c\}$ là một phản chuỗi dài hơn, mâu thuẫn.
2. b là một cạnh trên biên phải của một miền F_2 . Khi đó tồn tại một cạnh d trên biên trái của F_2 không so sánh được với cả a và b . Vậy $A \cup \{d\}$ là một phản chuỗi dài hơn, mâu thuẫn.
3. a nằm trên đường biên phải của miền F_1 , còn b nằm trên đường biên trái của miền F_2 . Đặt

$$a = (u_1, v_1), \quad b = (u_2, v_2).$$

Vì trong đồ thị phẳng đều tồn tại đường đi từ đỉnh cao nhất v_h tới u_1 và u_2 , giữa các đường đi ấy tồn tại ít nhất một điểm chung có thể đi tới cả u_1 và u_2 . Gọi điểm có tung độ thấp nhất trong các điểm chung ấy là u_l . Tương tự, gọi v_l là điểm có tung độ cao nhất trong các điểm chung có thể được đi tới từ cả v_1 và v_2 . Vì a và b không nằm trong cùng một miền, nên giữa đường 1

$$u_h \rightarrow \dots \rightarrow u_1 \rightarrow v_1 \rightarrow \dots \rightarrow u_l$$

và đường 2

$$u_h \rightarrow \dots \rightarrow u_2 \rightarrow v_2 \rightarrow \dots \rightarrow u_l$$

tồn tại ít nhất một đường thứ ba. Đường thứ ba này hoặc nối từ đường 1 sang đường 2, hoặc nối từ đường 2 sang đường 1. Vì vậy trên đường thứ ba nhất định tồn tại một cạnh e không so sánh được với a và b , suy ra $A \cup \{e\}$ là một phản chuỗi dài hơn, mâu thuẫn.

Do đó hai cạnh kề nhau trong phản chuỗi nhất định nằm trong cùng một miền.

A.3 Đề gốc: Skiers

Trên sườn nam của Bytemount có một số đường trượt tuyết và một thang kéo. Mọi đường trượt đều chạy từ ga trên của thang kéo xuống ga dưới. Mỗi buổi sáng, một nhóm nhân viên thang kéo kiểm tra tình trạng các đường trượt. Họ cùng đi thang kéo lên ga trên, rồi mỗi người trượt xuống theo một đường đã chọn tới ga dưới. Mỗi nhân viên chỉ trượt xuống một lần. Các đường mà nhân viên đi có thể trùng nhau một phần. Mỗi đường trượt được kiểm tra luôn đi xuống.

Bản đồ các đường trượt gồm một mạng các khoảng rừng trống nối với nhau bằng các lối cắt trong rừng. Mỗi khoảng rừng trống nằm ở một độ cao khác nhau. Hai khoảng rừng bất kỳ nhiều nhất được nối trực tiếp bởi một lối cắt. Khi trượt từ ga trên xuống ga dưới, có thể chọn một đường trượt để thăm bất kỳ một khoảng rừng trống nào, mặc dù có lẽ không thể thăm tất cả trong một lần trượt. Các đường trượt chỉ có thể cắt nhau tại các khoảng rừng trống, không đi qua đường hầm hay cầu.

Nhiệm vụ. Viết chương trình đọc bản đồ đường trượt từ tệp `nar.in`, tính số nhân viên tối thiểu cần có để kiểm tra mọi lối cắt giữa các khoảng rừng trống, rồi ghi kết quả ra tệp `nar.out`.

Dữ liệu vào. Dòng đầu tiên của tệp vào `nar.in` có một số nguyên n , là số khoảng rừng trống, $2 \leq n \leq 5000$. Các khoảng rừng được đánh số từ 1 tới n .

Mỗi dòng trong $n - 1$ dòng tiếp theo chứa một dãy số nguyên cách nhau bằng một dấu cách. Các số nguyên ở dòng thứ $i + 1$ của tệp chỉ ra các khoảng rừng trống mà những lối cắt từ khoảng rừng số i dẫn xuống. Số nguyên đầu tiên k chỉ số lượng các khoảng rừng ấy. k số nguyên tiếp theo là số hiệu của chúng, được sắp theo hướng từ tây sang đông, tương ứng với cách bố trí các lối cắt dẫn tới chúng. Ga trên của thang kéo nằm ở khoảng rừng số 1, còn ga dưới nằm ở khoảng rừng số n .

Dữ liệu ra. Dòng đầu tiên và duy nhất của tệp ra `nar.out` phải chứa đúng một số nguyên: số nhân viên tối thiểu có thể kiểm tra mọi lối cắt trong rừng.

A.4 Đề gốc: Alice and Bob

Bài toán. Đây là một câu đố cho hai người, gọi là Alice và Bob. Alice vẽ một đa giác lồi có n đỉnh và đánh số các đỉnh bằng các số nguyên $1, 2, \dots, n$ theo một cách tùy ý. Sau đó cô vẽ một số đường chéo không cắt nhau; các đỉnh của đa giác không được xem là điểm cắt. Cô cho Bob biết các cạnh và đường chéo của đa giác, nhưng không nói đường nào là đường nào. Mỗi cạnh và đường chéo được xác định bởi hai đầu mút. Bob phải đoán thứ tự các đỉnh trên biên đa giác. Hãy giúp anh ấy giải câu đố.

Ví dụ. Nếu $n = 4$ và $(1, 3), (4, 2), (1, 2), (4, 1), (2, 3)$ là các đầu mút của bốn cạnh và một đường chéo, thì thứ tự các đỉnh trên biên đa giác là

1, 3, 2, 4

chính xác tới phép quay vòng và đảo ngược.

Nhiệm vụ. Viết chương trình cho mỗi bộ dữ liệu:

- đọc mô tả các cạnh và đường chéo mà Alice đưa cho Bob;
- tính thứ tự các đỉnh trên biên đa giác;

- ghi kết quả.

Dữ liệu vào. Dòng đầu tiên của dữ liệu vào chứa đúng một số nguyên dương d , là số bộ dữ liệu, $1 \leq d \leq 20$. Sau đó là các bộ dữ liệu.

Mỗi bộ dữ liệu gồm đúng hai dòng liên tiếp. Dòng đầu tiên chứa đúng hai số nguyên n và m , cách nhau bởi một dấu cách, trong đó $3 \leq n \leq 10000$ và $0 \leq m \leq n - 3$. Số nguyên n là số đỉnh của đa giác, còn m là số đường chéo.

Dòng thứ hai chứa đúng $2(m + n)$ số nguyên cách nhau bởi dấu cách. Đó là các đầu mút của mọi cạnh và một số đường chéo của đa giác. Các số nguyên a_j, b_j ở vị trí $2j - 1$ và $2j$, với $1 \leq j \leq m + n$,

$$1 \leq a_j \leq n, \quad 1 \leq b_j \leq n, \quad a_j \neq b_j,$$

xác định hai đầu mút của một cạnh hoặc một đường chéo. Các cạnh và đường chéo có thể được cho theo thứ tự tùy ý. Không có cạnh trùng. Alice không gian lận, tức câu đố luôn có nghiệm.

Dữ liệu ra. Dữ liệu ra gồm đúng d dòng, mỗi dòng ứng với một bộ dữ liệu. Dòng thứ i , $1 \leq i \leq d$, phải chứa một dãy n số nguyên cách nhau bởi dấu cách: một hoán vị của $1, 2, \dots, n$, tức các số hiệu đỉnh liên tiếp trên biên của đa giác trong bộ dữ liệu thứ i . Dãy phải luôn bắt đầu bằng 1, và phần tử thứ hai phải là đỉnh nhỏ hơn trong hai đỉnh kề với đỉnh 1 trên biên.